Problem Statement 11: NinjaShop – Selenium-Java Test Automation Framework for a Full-Featured Demo Online Store

---

1. Hackathon Objective

This hackathon evaluates students on:

- *Selenium WebDriver fundamentals and browser automation*
- *Page Object Model (POM) design pattern*
- *TestNG framework – annotations, listeners, and DataProviders*
- *Explicit wait strategy for handling dynamic elements*
- *Screenshot capture on test failure*
- *Clean, reusable, and maintainable framework design*

---

2. Problem Statement

Design and develop a Selenium-Java Automation Test Framework for a live full-featured demo e-commerce web application.

The framework should automate end-to-end test scenarios across the following functional areas:

- *User registration, login, and logout*
- *Product search and detail verification*
- *Shopping cart management*
- *Checkout address and order placement*
- *Form validations – empty and invalid inputs*

The framework must follow the Page Object Model pattern and must be structured as a reusable framework — not just a collection of standalone test scripts.

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Confidential

3. Application Under Test (AUT)

URL: https://tutorialsninja.com/demo

No backend setup required. Students directly automate this live demo store.

Key modules available for testing:

- ***Auth** – Register, Login, Logout*
- ***Products** – Search, Category Browse, Product Detail*
- ***Cart** – Add to Cart, Update, Remove*
- ***Checkout** – Delivery Details, Confirm Order*
- ***My Account** – View Order History, Edit Profile*

4. Test Modules and Scope

4.1 Test Module 1 – User Authentication

- *Verify successful login with valid credentials*
- *Verify login failure with wrong password and validate the error message*
- *Verify logout redirects to the home page*
- *Verify registration with a new unique email and valid data is successful*

4.2 Test Module 2 – Product Search and Browse

- *Search for a product by keyword and verify results are displayed*
- *Navigate to a category from the top navigation and verify products are listed*
- *Open a product detail page and verify the product name and price are shown*
- *Verify that searching for a nonexistent product shows a no-results message*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

4.3 Test Module 3 – Shopping Cart

- *Add a product to cart and verify it appears with the correct name and price*
- *Update the quantity of a product in the cart and verify the line total changes*
- *Remove a product from the cart and verify the cart shows as empty*
- *Verify the cart icon in the header reflects the correct item count*

4.4 Test Module 4 – Checkout Flow

- *Proceed to checkout as a logged-in user and fill in valid delivery details*
- *Confirm the order and verify the order confirmation message is shown*
- *Attempt to checkout without being logged in and verify redirect to the login page*

4.5 Test Module 5 – Form Validations

- *Submit the registration form with empty fields and verify error messages for each field*
- *Enter an invalid email format during registration and verify the validation*
- *Submit checkout with missing delivery address fields and verify field-level errors*

---

5. Must-Have Requirements

5.1 Page Object Model (POM)

- *Every page must have a dedicated Page class (LoginPage, RegisterPage, ProductPage, CartPage, CheckoutPage)*
- *Locators and action methods must be inside Page classes only*
- *Test classes must only call Page class methods — no findElement or click in test classes*
- *A BasePage class must have at least one shared utility (example: waitForElement)*

5.2 TestNG Configuration

- *testng.xml must define the test suite and list all test classes*
- *Use @BeforeMethod and @AfterMethod for browser launch and teardown*
- *Use at least one @DataProvider for login tests (valid and invalid credentials)*

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Confidential

5.3 WebDriver Management

- *Use WebDriverManager — no manual driver setup*
- *Browser name must be read from config.properties (chrome or firefox)*

5.4 Wait Strategy

- *Thread.sleep() is strictly not allowed — deduction will be applied*
- *Use WebDriverWait with ExpectedConditions for elements that load dynamically*

5.5 Configuration Management

- *config.properties must hold: browser, base URL, and timeout value*
- *A ConfigReader class must read and provide these values*
- *No hardcoded URLs or values in any test or page class*

5.6 Screenshot on Failure

- *Implement ITestListener to capture a screenshot automatically when a test fails*
- *Save screenshots with a timestamp in the filename inside a /screenshots/ folder*

5.7 Test Reporting

- *ExtentReports must generate an HTML report after every run*
- *Report must show test name, pass/fail status, and screenshot on failure*

Required Maven dependencies: *selenium-java, testng, webdrivermanager, extentreports*

---

6. Business Rules (Strictly Enforced)

- *Thread.sleep() anywhere results in a score deduction*
- *Hardcoded URLs or credentials in any class other than config.properties results in a deduction*
- *WebDriver code directly inside test methods results in a POM score deduction*
- *Suite must run from testng.xml using: mvn test*

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Confidential

7. Good to Have

- *@DataProvider reading from a JSON or Excel file instead of inline data*
- *Parallel execution in testng.xml (thread-count="2")*
- *FluentWait for slow-loading elements*
- *Headless browser mode from config.properties*
- *Retry on failure using IRetryAnalyzer*

8. Non-Functional Expectations

- *Clean separation between Page classes, Test classes, and config*
- *No duplicate code — shared actions must go into BasePage or utilities*
- *Meaningful method and variable names throughout*
- *No Thread.sleep() — all timing handled via WebDriverWait*
- *Code readability over quantity — a clean small framework beats a messy large one*

9. Hackathon Guidelines

- *Spend the first 20–30 minutes planning before writing any code*
- *Internet and AI usage is allowed — but every member must explain their own code*
- *Every member must contribute at least one Page class and one Test class*
- *No hardcoded URLs or credentials anywhere except config.properties*
- *Final walkthrough: 5 minutes per team, each member speaks about their own work*

10. Evaluation Criteria

- *Framework Design*

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Confidential

- *Test Coverage*
- *Selenium Proficiency*
- *Data-Driven and Config Design*
- *Reporting and Evidence*
- *Team Collaboration*

Confidential

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.